

STRUCTURED P2P GOSSIP FOR SCALABLE OVERLAYS:A CASE FOR CYCLON GOSSIP PROTOCOL

Project report

Submitted by

Sakshee Ujjwal Kumat , 23BCE1238

M Menmangai , 23BCE1022

Sanjana Raghunath, 23BCE1426

COMPUTER NETWORKS

Dr. M. SIVAGAMI

BCSE308L

C1+TC1

CH2024250502014

Winter Semester 2024-2025

ABSTRACT

This project aims to understand, implement, and simulate the Gossip protocol using the C programming language, and analyze it in terms of message redundancy and reachability as the number of nodes increases in a given peer-to-peer(P2P) network. The fundamental principle behind gossip-based communication is that in a decentralized network, each node periodically selects a few random peers and shares known messages with them. Over time, this ensures that all nodes in the network eventually receive the message, a property known as eventual consistency. This protocol is widely used in databases, one such example is Apache Cassandra, a NoSQL database for node discovery, failure detection and state dissemination. While gossiping is significantly more scalable and fault-tolerant than naive broadcasting where each node blindly sends messages to all others, it still suffers from message redundancy. Since peer selection is random, the same node may receive the same message multiple times, leading to inefficient bandwidth usage and unnecessary network traffic. To address this limitation, a more structured variant known as Cyclon Gossip was studied and implemented. Cyclon introduces the concept of structured peer sampling, where each node maintains a partial, constantly shuffled view of the network. This leads to more diverse and uniform communication patterns, reducing redundancy, improving message reachability, and ensuring faster message dissemination across the network. Both the traditional gossip and the Cyclon-enhanced protocol were simulated in C and tested on networks with varying sizes. Graphs and equations were derived for the normal gossip protocol to get the number of redundant messages as the number of nodes in the network increases and from the graph, equations were appropriately derived. The study concludes that Cyclon significantly improves gossip efficiency, making it more suitable for large-scale and resource-constrained P2P systems.

Keywords: Gossip,peer-to-peer,Cyclon,Apache Cassandra, reachability,
redundancy,graph

MODULES OF THE PROJECT

Modules of the Cyclon Gossip Protocol Implementation

Core Modules

1. Node Management

- **NodeDescriptor Structure:**
 - Manages node identity with unique ID, IP address, port, and timestamp
 - Acts as the fundamental unit of peer information
 - Provides context for network topology discovery
- **View Management:**
 - Maintains local partial view of network peers (limited to VIEW_LENGTH)
 - Implements the unstructured overlay network concept
 - Provides resilience through redundant connections
- **View Manipulation Functions:**
 - add_descriptor(): Adds new peers if space available
 - remove_descriptor(): Removes peers by index
 - update_descriptor(): Updates existing or adds new peers
 - find_oldest_descriptor(): Identifies longest-known peer for cycling

2. Cyclon Protocol Implementation

- **Gossip Cycle Management:**
 - Time-based triggering (every 10 seconds)

- Incremental peer set refreshing for network topology evolution
- Maintains peer selection history to prevent consecutive selections
- **Node Selection Algorithms:**
 - Age-based selection (oldest first) for cycling
 - Random sampling with Fisher-Yates shuffle for subset selection
 - `select_random_descriptors()`: Implements randomized peer selection
- **Descriptor Exchange:**
 - Push-pull mechanism for peer information exchange
 - `SWAP_LENGTH` parameter controls exchange volume
 - Timestamp refreshing ensures information freshness

3. Message Handling

- **Protocol Message Parsing:**
 - Parses structured messages (`CYCLON_PUSH`, `CYCLON_REPLY`)
 - Extracts descriptor information from message payloads
 - Handles regular gossip messages with sender information
- **Message Caching:**
 - `cached_msgs` array stores recently seen messages
 - `is_duplicate_message()` detects repeated messages
 - FIFO replacement policy for cache management
- **Message Forwarding:**
 - `FORWARD_COUNT` parameter limits propagation factor

- Random peer selection for gossip dissemination
- Epidemic spreading pattern for network-wide coverage

4. Network Communication

- **Socket Management:**

- UDP datagram-based communication
- Non-connection-oriented messaging for scalability
- Address resolution and message transmission

- **I/O Multiplexing:**

- select() function monitors multiple input sources
- Handles both network events and user input
- Timeout-based cycling for protocol operations

- **User Interaction:**

- Command processing (VIEW, CYCLE, BYE)
- User message input and formatting
- Status reporting and view display

TECHNOLOGIES USED

1. Programming Language: C

2. Core Libraries:

- Standard C libraries: `stdio.h`, `stdlib.h`, `string.h`, `time.h`
- POSIX networking: `unistd.h`, `sys/socket.h`, `sys/types.h`
- Network protocols: `arpa/inet.h`, `netinet/in.h`
- File descriptor management: `fcntl.h`

3. Communication Protocol:

UDP (User Datagram Protocol)

4. System Architecture:

- Distributed peer-to-peer network
- Event-driven programming model
- Non-blocking I/O with `select()`

5. Algorithms:

- Fisher-Yates shuffle for random peer selection
- FIFO cache for message deduplication
- View shuffling via the Cyclon gossip protocol

This implementation provides a lightweight, decentralized membership protocol that enables peer discovery and maintenance in distributed systems without requiring central coordination.

SOURCE CODES

i)Traditional Gossip Protocol Implementation:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>      // For sleep()
#include <arpa/inet.h>
#include <sys/socket.h>
#include <sys/types.h>
#include <netinet/in.h>
#include <fcntl.h>
#include <time.h>

#define MAX_BUFFER_SIZE 1024
#define MAX_USERS 100
#define GOSSIP_FANOUT 2 // Send to 2 peers at once

typedef struct {
    char ipaddr[50];
    int port;
} Detail;

void error(const char *msg) {
    perror(msg);
    exit(EXIT_FAILURE);
}
```

```

int is_duplicate(char *msg, char received_msgs[][MAX_BUFFER_SIZE], int
count) {
    for (int i = 0; i < count; i++) {
        if (strcmp(msg, received_msgs[i]) == 0) {
            return 1;
        }
    }
    return 0;
}

```

```

void select_random_peers(int *selected, int usercount, int exclude) {
    int used[MAX_USERS] = {0}, selected_count = 0;
    srand(time(NULL));

    while (selected_count < GOSSIP_FANOUT && selected_count < usercount -
1) {
        int rand_index = rand() % usercount;
        if (rand_index != exclude && !used[rand_index]) {
            selected[selected_count++] = rand_index;
            used[rand_index] = 1;
        }
    }
}

```

```

int main(int argc, char *argv[]) {
    if (argc < 2) {
        fprintf(stderr, "Usage: %s <port>\n", argv[0]);
        exit(EXIT_FAILURE);
    }
}

```

```

}

int srvsock, portno, maxfd, result;
struct sockaddr_in serveraddr, clientaddr;
socklen_t len = sizeof(clientaddr);
fd_set readset, tempset;
char buf[MAX_BUFFER_SIZE],
received_msgs[MAX_USERS][MAX_BUFFER_SIZE];
int received_count = 0;

FILE *userFile = fopen("users.txt", "r");
if (!userFile) error("Error opening users.txt");

Detail users[MAX_USERS];
char usernames[MAX_USERS][50];
int usercount = 0;

while (fscanf(userFile, "%s %s %d", usernames[usercount],
users[usercount].ipaddr, &users[usercount].port) == 3) {
    usercount++;
}
fclose(userFile);

portno = atoi(argv[1]);
srvsock = socket(AF_INET, SOCK_DGRAM, 0);
if (srvsock < 0) error("ERROR opening socket");

memset(&serveraddr, 0, sizeof(serveraddr));

```

```

serveraddr.sin_family = AF_INET;
serveraddr.sin_addr.s_addr = INADDR_ANY;
serveraddr.sin_port = htons(portno);

if (bind(srvsock, (struct sockaddr *)&serveraddr, sizeof(serveraddr)) < 0) {
    error("ERROR on binding");
}

char myName[50];
int myIndex = -1;
for (int i = 0; i < usercount; i++) {
    if (users[i].port == portno) {
        strcpy(myName, usernames[i]);
        myIndex = i;
        break;
    }
}

if (myIndex == -1) error("No matching user found for the provided port");

FD_ZERO(&readset);
FD_SET(srvsock, &readset);
FD_SET(STDIN_FILENO, &readset);
maxfd = srvsock;

while (1) {
    tempset = readset;
    result = select(maxfd + 1, &tempset, NULL, NULL, NULL);
}

```

```

if (result < 0) error("ERROR on select");

if (FD_ISSET(srvsock, &tempset)) {
    memset(buf, 0, MAX_BUFFER_SIZE);
    recvfrom(srvsock, buf, MAX_BUFFER_SIZE, 0, (struct sockaddr
*)&clientaddr, &len);

    if (!is_duplicate(buf, received_msgs, received_count)) {
        strcpy(received_msgs[received_count++], buf);
        printf("\n[GOSSIP RECEIVED] %s\n", buf);

        int selected_peers[GOSSIP_FANOUT];
        select_random_peers(selected_peers, usercount, myIndex);

        struct sockaddr_in peeraddr;

        // Send to 2 peers simultaneously
        printf("→ Sending to peers:\n");
        for (int i = 0; i < GOSSIP_FANOUT; i++) {
            memset(&peeraddr, 0, sizeof(peeraddr));
            peeraddr.sin_family = AF_INET;
            peeraddr.sin_port = htons(users[selected_peers[i]].port);
            inet_pton(AF_INET, users[selected_peers[i]].ipaddr,
&peeraddr.sin_addr);

            printf("    → Peer: %s:%d\n", users[selected_peers[i]].ipaddr,
users[selected_peers[i]].port);

            sendto(srvsock, buf, strlen(buf), 0, (struct sockaddr *)&peeraddr,
sizeof(peeraddr));

```

```

    }

    // Sleep for delay before sending to the next peers
    sleep(2);
}
}

if (FD_ISSET(STDIN_FILENO, &tempset)) {
    memset(buf, 0, MAX_BUFFER_SIZE);
    read(STDIN_FILENO, buf, MAX_BUFFER_SIZE);
    buf[strcspn(buf, "\n")] = 0;

    if (strcmp(buf, "BYE") == 0) {
        printf("Exiting...\n");
        break;
    }

    char formattedMessage[MAX_BUFFER_SIZE];
    snprintf(formattedMessage, MAX_BUFFER_SIZE, "%s: %.900s",
myName, buf);

    strcpy(received_msgs[received_count++], formattedMessage);
    printf("\n[GOSSIP SENT] %s\n", formattedMessage);

    int selected_peers[GOSSIP_FANOUT];
    select_random_peers(selected_peers, usercount, myIndex);

    struct sockaddr_in peeraddr;

```

```

// Send to 2 peers simultaneously
printf("→ Sending to peers:\n");
for (int i = 0; i < GOSSIP_FANOUT; i++) {
    memset(&peeraddr, 0, sizeof(peeraddr));
    peeraddr.sin_family = AF_INET;
    peeraddr.sin_port = htons(users[selected_peers[i]].port);
    inet_pton(AF_INET, users[selected_peers[i]].ipaddr,
&peeraddr.sin_addr);

    printf("    → Peer: %s:%d\n", users[selected_peers[i]].ipaddr,
users[selected_peers[i]].port);
    sendto(srvsock, formattedMessage, strlen(formattedMessage), 0,
(struct sockaddr *)&peeraddr, sizeof(peeraddr));
}

// Sleep before sending to the next peers
sleep(2);
}
}

close(srvsock);
return 0;
}

```

OUTPUT:

```
sakshee@LAPTOP-7PLB9TI2:~$ gcc p2p_gossip_protocol_delay.c -o peer
sakshee@LAPTOP-7PLB9TI2:~$ ./peer 5000
Hello

[GOSSIP SENT] Alice: Hello
→ Sending to peers:
  → Peer: 127.0.0.1:5001
  → Peer: 127.0.0.1:5002

sakshee@LAPTOP-7PLB9TI2:~$ gcc p2p_gossip_protocol_delay.c -o peer
sakshee@LAPTOP-7PLB9TI2:~$ ./peer 5002

[GOSSIP RECEIVED] Alice: Hello
→ Sending to peers:
  → Peer: 127.0.0.1:5001
  → Peer: 127.0.0.1:5003

sakshee@LAPTOP-7PLB9TI2:~$ gcc p2p_gossip_protocol_delay.c -o peer
sakshee@LAPTOP-7PLB9TI2:~$ ./peer 5001

[GOSSIP RECEIVED] Alice: Hello
→ Sending to peers:
  → Peer: 127.0.0.1:5002
  → Peer: 127.0.0.1:5003

sakshee@LAPTOP-7PLB9TI2:~$ gcc p2p_gossip_protocol_delay.c -o peer
sakshee@LAPTOP-7PLB9TI2:~$ ./peer 5003

[GOSSIP RECEIVED] Alice: Hello
→ Sending to peers:
  → Peer: 127.0.0.1:5001
  → Peer: 127.0.0.1:5002

sakshee@LAPTOP-7PLB9TI2:~$ gcc p2p_gossip_protocol_delay.c -o peer
sakshee@LAPTOP-7PLB9TI2:~$ ./peer 5000
Hello

[GOSSIP SENT] Alice: Hello
→ Sending to peers:
  → Peer: 127.0.0.1:5001
  → Peer: 127.0.0.1:5002

[GOSSIP RECEIVED] Dave: heyyy everyone I am good
→ Sending to peers:
  → Peer: 127.0.0.1:5002
  → Peer: 127.0.0.1:5001

sakshee@LAPTOP-7PLB9TI2:~$ gcc p2p_gossip_protocol_delay.c -o peer
sakshee@LAPTOP-7PLB9TI2:~$ ./peer 5001

[GOSSIP RECEIVED] Alice: Hello
→ Sending to peers:
  → Peer: 127.0.0.1:5002
  → Peer: 127.0.0.1:5003

[GOSSIP RECEIVED] Dave: heyyy everyone I am good
→ Sending to peers:
  → Peer: 127.0.0.1:5002
  → Peer: 127.0.0.1:5000

sakshee@LAPTOP-7PLB9TI2:~$ gcc p2p_gossip_protocol_delay.c -o peer
sakshee@LAPTOP-7PLB9TI2:~$ ./peer 5002

[GOSSIP RECEIVED] Alice: Hello
→ Sending to peers:
  → Peer: 127.0.0.1:5001
  → Peer: 127.0.0.1:5002
heyyy everyone I am good

[GOSSIP SENT] Dave: heyyy everyone I am good
→ Sending to peers:
  → Peer: 127.0.0.1:5002
  → Peer: 127.0.0.1:5000
```

ii) Cyclon-enhanced Gossip Protocol Implementation:

gossip_cyclon.c:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/socket.h>
#include <sys/types.h>
#include <netinet/in.h>
#include <fcntl.h>
#include <time.h>

#define MAX_BUFFER_SIZE 1024
#define MAX_USERS 100
#define MAX_CACHED_MSGS 50
#define VIEW_LENGTH 3
#define SWAP_LENGTH 2
#define FORWARD_COUNT 2

typedef struct {
    char id[50];
    char ipaddr[50];
    int port;
    time_t timestamp;
} NodeDescriptor;
```

```

typedef struct {
    NodeDescriptor descriptors[VIEW_LENGTH];
    int count;
} View;

void error(const char *msg) {
    perror(msg);
    exit(EXIT_FAILURE);
}

int find_oldest_descriptor(View *view) {
    if (view->count == 0) return -1;

    int oldest_idx = 0;
    time_t oldest_time = view->descriptors[0].timestamp;

    for (int i = 1; i < view->count; i++) {
        if (view->descriptors[i].timestamp < oldest_time) {
            oldest_time = view->descriptors[i].timestamp;
            oldest_idx = i;
        }
    }

    return oldest_idx;
}

// Remove a descriptor at specified index from the view

```

```
NodeDescriptor remove_descriptor(View *view, int index) {  
    if (index < 0 || index >= view->count) {  
        NodeDescriptor empty;  
        memset(&empty, 0, sizeof(NodeDescriptor));  
        return empty;  
    }  
}
```

```
NodeDescriptor removed = view->descriptors[index];
```

```
// Shift remaining elements
```

```
for (int i = index; i < view->count - 1; i++) {  
    view->descriptors[i] = view->descriptors[i + 1];  
}
```

```
view->count--;
```

```
return removed;
```

```
}
```

```
// Add a descriptor to the view if there's space
```

```
int add_descriptor(View *view, NodeDescriptor descriptor) {
```

```
    // Don't add if view is full
```

```
    if (view->count >= VIEW_LENGTH) {
```

```
        return 0;
```

```
    }
```

```
// Don't add descriptors with empty id
```

```
if (strlen(descriptor.id) == 0) {
```

```

    return 0;
}

// Check if already exists
for (int i = 0; i < view->count; i++) {
    if (strcmp(view->descriptors[i].id, descriptor.id) == 0) {
        // Update timestamp if it already exists
        view->descriptors[i].timestamp = descriptor.timestamp;
        return 0; // Already exists (but timestamp updated)
    }
}

view->descriptors[view->count++] = descriptor;
return 1;
}

// Update or add descriptor in view
int update_descriptor(View *view, NodeDescriptor descriptor) {
    // Don't add descriptors with empty id
    if (strlen(descriptor.id) == 0) {
        return 0;
    }

    // Check if already exists
    for (int i = 0; i < view->count; i++) {
        if (strcmp(view->descriptors[i].id, descriptor.id) == 0) {
            // Update timestamp

```

```

        view->descriptors[i].timestamp = descriptor.timestamp;
        return 1; // Updated existing
    }
}

// Add if not exists and we have space
if (view->count < VIEW_LENGTH) {
    view->descriptors[view->count++] = descriptor;
    return 1;
}

return 0;
}

// Select random descriptors from view (and remove them)
int select_random_descriptors(View *view, NodeDescriptor *selected, int count)
{
    if (view->count == 0) return 0;

    int selected_count = 0;
    int indices[VIEW_LENGTH];
    int available = view->count;

    // Initialize indices array
    for (int i = 0; i < view->count; i++) {
        indices[i] = i;
    }
}

```

```

// Don't try to select more than what's available
count = (count < available) ? count : available;

// Fisher-Yates shuffle to select random indices
for (int i = 0; i < count && available > 0; i++) {
    int j = rand() % available;
    selected[selected_count++] = view->descriptors[indices[j]];

    // Remove the selected descriptor
    for (int k = indices[j]; k < view->count - 1; k++) {
        view->descriptors[k] = view->descriptors[k + 1];
    }
    view->count--;

    // Update indices array
    indices[j] = indices[available - 1];
    available--;
}

return selected_count;
}

// Check if a message has been seen before
int is_duplicate_message(char *msg, char
cached_msgs[][MAX_BUFFER_SIZE], int *msg_count) {
    for (int i = 0; i < *msg_count; i++) {
        if (strcmp(msg, cached_msgs[i]) == 0) {
            return 1;
        }
    }
}

```

```

    }
}

// Add to cache if not found
if (*msg_count < MAX_CACHED_MSGS) {
    strcpy(cached_msgs[*msg_count], msg);
    (*msg_count)++;
} else {
    // Replace oldest message (simple FIFO)
    for (int i = 0; i < MAX_CACHED_MSGS - 1; i++) {
        strcpy(cached_msgs[i], cached_msgs[i+1]);
    }
    strcpy(cached_msgs[MAX_CACHED_MSGS-1], msg);
}

return 0;
}

int main(int argc, char *argv[]) {
    if (argc < 2) {
        fprintf(stderr, "Usage: %s <port>\n", argv[0]);
        exit(EXIT_FAILURE);
    }

    int srvsock, portno, maxfd, result;
    struct sockaddr_in serveraddr, clientaddr;
    socklen_t len = sizeof(clientaddr);

```

```

fd_set readset, tempset;
char buf[MAX_BUFFER_SIZE];
char cached_msgs[MAX_CACHED_MSGS][MAX_BUFFER_SIZE];
int msg_count = 0;

// Remember the last gossip partner to avoid repeat exchanges
NodeDescriptor last_partner;
memset(&last_partner, 0, sizeof(NodeDescriptor));

// Initialize random seed
srand(time(NULL) ^ getpid());

// Read users from file
FILE *userFile = fopen("users.txt", "r");
if (!userFile) error("Error opening users.txt");

NodeDescriptor allUsers[MAX_USERS];
int userCount = 0;

while (fscanf(userFile, "%s %s %d",
              allUsers[userCount].id,
              allUsers[userCount].ipaddr,
              &allUsers[userCount].port) == 3) {
    allUsers[userCount].timestamp = time(NULL);
    userCount++;
}
fclose(userFile);

```

```
if (userCount < 2) {
    error("Need at least 2 users in users.txt");
}

// Find my own descriptor
portno = atoi(argv[1]);
NodeDescriptor myDescriptor;
int myIndex = -1;

for (int i = 0; i < userCount; i++) {
    if (allUsers[i].port == portno) {
        myDescriptor = allUsers[i];
        myIndex = i;
        break;
    }
}

if (myIndex == -1) error("No matching user found for the provided port");

// Initialize socket
srvsock = socket(AF_INET, SOCK_DGRAM, 0);
if (srvsock < 0) error("ERROR opening socket");

memset(&serveraddr, 0, sizeof(serveraddr));
serveraddr.sin_family = AF_INET;
serveraddr.sin_addr.s_addr = INADDR_ANY;
```

```

serveraddr.sin_port = htons(portno);

if (bind(srvsock, (struct sockaddr *)&serveraddr, sizeof(serveraddr)) < 0) {
    error("ERROR on binding");
}

// Initialize my view with RANDOM subset of nodes (proper bootstrapping)
View myView;
myView.count = 0;

// Create array of indices excluding myself
int otherIndices[MAX_USERS];
int otherCount = 0;
for (int i = 0; i < userCount; i++) {
    if (i != myIndex) {
        otherIndices[otherCount++] = i;
    }
}

// Shuffle the indices
for (int i = otherCount - 1; i > 0; i--) {
    int j = rand() % (i + 1);
    int temp = otherIndices[i];
    otherIndices[i] = otherIndices[j];
    otherIndices[j] = temp;
}

```

```

// Add at most VIEW_LENGTH random nodes (excluding myself)
int initialViewSize = (otherCount < VIEW_LENGTH) ? otherCount :
VIEW_LENGTH;
for (int i = 0; i < initialViewSize; i++) {
    add_descriptor(&myView, allUsers[otherIndices[i]]);
}

printf("Node %s initialized with %d nodes in view\n", myDescriptor.id,
myView.count);

// Display initial view
printf("Initial view contents:\n");
for (int i = 0; i < myView.count; i++) {
    printf(" %d. %s (%s:%d)\n",
        i+1,
        myView.descriptors[i].id,
        myView.descriptors[i].ipaddr,
        myView.descriptors[i].port);
}

// Set up for select()
FD_ZERO(&readset);
FD_SET(srvsock, &readset);
FD_SET(STDIN_FILENO, &readset);
maxfd = (srvsock > STDIN_FILENO) ? srvsock : STDIN_FILENO;

// Set up cyclic timer for Cyclon protocol (every 10 seconds)
time_t last_cycle_time = time(NULL);

```

```

int cycle_interval = 10; // seconds

while (1) {
    // Set timeout for select to handle cyclic behavior
    struct timeval tv;
    tv.tv_sec = 1; // Check every second
    tv.tv_usec = 0;

    tempset = readset;
    result = select(maxfd + 1, &tempset, NULL, NULL, &tv);

    if (result < 0) error("ERROR on select");

    // Check if it's time for a Cyclon cycle
    time_t current_time = time(NULL);
    if ((current_time - last_cycle_time) >= cycle_interval) {
        last_cycle_time = current_time;

        if (myView.count > 0) {
            printf("\n[CYCLON CYCLE] Initiating gossip exchange\n");

            // Step 1: Select oldest node from view
            int oldest_idx = find_oldest_descriptor(&myView);
            if (oldest_idx >= 0) {
                NodeDescriptor partner = remove_descriptor(&myView,
oldest_idx);

                // Avoid selecting the same partner twice in a row

```

```

if (strcmp(partner.id, last_partner.id) == 0 && myView.count > 0) {
    // Put this descriptor back and get next oldest
    add_descriptor(&myView, partner);
    oldest_idx = find_oldest_descriptor(&myView);
    partner = remove_descriptor(&myView, oldest_idx);
}

// Save this partner as the last one selected
last_partner = partner;

printf("→ Selected gossip partner: %s:%d\n", partner.id,
partner.port);

// Step 2: Select descriptors to send
NodeDescriptor to_send[SWAP_LENGTH];
int sendable = SWAP_LENGTH - 1; // Reserve one slot for self
int random_count = 0;

if (sendable > 0 && myView.count > 0) {
    // Select random descriptors from view
    random_count = select_random_descriptors(&myView,
&to_send[1], sendable);
}

// First descriptor is always a fresh descriptor of myself
myDescriptor.timestamp = time(NULL); // Update timestamp
to_send[0] = myDescriptor;

```

```

// Step 3: Send descriptors to partner
struct sockaddr_in peeraddr;
memset(&peeraddr, 0, sizeof(peeraddr));
peeraddr.sin_family = AF_INET;
peeraddr.sin_port = htons(partner.port);
inet_pton(AF_INET, partner.ipaddr, &peeraddr.sin_addr);

// Create message with descriptors to send
char msg[MAX_BUFFER_SIZE];
memset(msg, 0, MAX_BUFFER_SIZE);

//Format:"CYCLON_PUSH:<count>:<id1>:<ip1>:<port1>:<timestamp1>:..."
int total_to_send = 1 + random_count; // self + random
char *ptr = msg;
ptr += sprintf(ptr, "CYCLON_PUSH:%d:", total_to_send);

for (int i = 0; i < total_to_send; i++) {
    ptr += sprintf(ptr, "%s:%s:%d:%ld:",
        to_send[i].id,
        to_send[i].ipaddr,
        to_send[i].port,
        to_send[i].timestamp);
}

printf("→ Sending %d descriptors to %s\n", total_to_send,
partner.id);

```

```

        sendto(srvsock, msg, strlen(msg), 0, (struct sockaddr *)&peeraddr,
sizeof(peeraddr));
    }
}
}

```

```

// Handle incoming messages
if (result > 0 && FD_ISSET(srvsock, &tempset)) {
    memset(buf, 0, MAX_BUFFER_SIZE);
    recvfrom(srvsock, buf, MAX_BUFFER_SIZE, 0, (struct sockaddr
*)&clientaddr, &len);

```

```

// Parse message
if (strncmp(buf, "CYCLON_PUSH:", 12) == 0) {
    // Another node initiated a gossip exchange with us
    printf("\n[CYCLON RECEIVED] Exchange request\n");

```

```

// Parse the message to extract descriptors
char *token = strtok(buf, ":");
token = strtok(NULL, ":"); // Skip "CYCLON_PUSH"

```

```

if (!token) continue; // Malformed message
int count = atoi(token);
NodeDescriptor received[VIEW_LENGTH];
int received_count = 0;
NodeDescriptor sender; // Keep track of who sent this request
memset(&sender, 0, sizeof(NodeDescriptor));

```

```

// Extract descriptors
for (int i = 0; i < count && received_count < VIEW_LENGTH; i++) {
    char id[50], ipaddr[50];
    int port;
    time_t timestamp;

    token = strtok(NULL, ":"); // id
    if (!token) break;
    strcpy(id, token);

    token = strtok(NULL, ":"); // ipaddr
    if (!token) break;
    strcpy(ipaddr, token);

    token = strtok(NULL, ":"); // port
    if (!token) break;
    port = atoi(token);

    token = strtok(NULL, ":"); // timestamp
    if (!token) break;
    timestamp = atol(token);

    if (strlen(id) > 0 && port > 0) {
        NodeDescriptor desc;
        strcpy(desc.id, id);
        strcpy(desc.ipaddr, ipaddr);
        desc.port = port;
    }
}

```

```

        // Always use current time for freshness
        desc.timestamp = time(NULL);

        received[received_count++] = desc;

        // The first descriptor is always the sender
        if (i == 0) {
            sender = desc;
        }
    }
}

// Step 4: Select descriptors to reply with
NodeDescriptor to_reply[SWAP_LENGTH];
int reply_count = 0;

// Select random descriptors from my view
if (myView.count > 0) {
    reply_count = select_random_descriptors(&myView, to_reply,
SWAP_LENGTH);
}

// Step 5: Add received descriptors to my view (excluding myself)
int added = 0;
for (int i = 0; i < received_count; i++) {
    if (strcmp(received[i].id, myDescriptor.id) != 0) {
        if (add_descriptor(&myView, received[i])) {
            added++;
        }
    }
}

```

```

    }
}
}

```

```
printf("→ Added %d descriptors to my view\n", added);
```

```
// Step 6: Send reply back
```

```
char reply[MAX_BUFFER_SIZE];
```

```
memset(reply, 0, MAX_BUFFER_SIZE);
```

```
//
```

Format:

```
"CYCLON_REPLY:<count>:<id1>:<ip1>:<port1>:<timestamp1>:..."
```

```
char *ptr = reply;
```

```
ptr += sprintf(ptr, "CYCLON_REPLY:%d:", reply_count);
```

```
for (int i = 0; i < reply_count; i++) {
```

```
    ptr += sprintf(ptr, "%s:%s:%d:%ld:",
```

```
        to_reply[i].id,
```

```
        to_reply[i].ipaddr,
```

```
        to_reply[i].port,
```

```
        to_reply[i].timestamp);
```

```
}
```

```
printf("→ Replying with %d descriptors\n", reply_count);
```

```
sendto(srvsock, reply, strlen(reply), 0, (struct sockaddr *)&clientaddr,
sizeof(clientaddr));
```

```
// Add the exchange partner back to view with updated timestamp
```

```

update_descriptor(&myView, sender);

} else if (strcmp(buf, "CYCLON_REPLY:", 13) == 0) {
    // Received reply to our gossip request
    printf("\n[CYCLON RECEIVED] Exchange reply\n");

    // Parse the message to extract descriptors
    char *token = strtok(buf, ":");
    token = strtok(NULL, ":"); // Skip "CYCLON_REPLY"

    if (!token) continue; // Malformed message
    int count = atoi(token);
    NodeDescriptor received[VIEW_LENGTH];
    int received_count = 0;

    // Extract descriptors
    for (int i = 0; i < count && received_count < VIEW_LENGTH; i++) {
        char id[50], ipaddr[50];
        int port;
        time_t timestamp;

        token = strtok(NULL, ":"); // id
        if (!token) break;
        strcpy(id, token);

        token = strtok(NULL, ":"); // ipaddr
        if (!token) break;

```

```
strcpy(ipaddr, token);
```

```
token = strtok(NULL, ":"); // port
```

```
if (!token) break;
```

```
port = atoi(token);
```

```
token = strtok(NULL, ":"); // timestamp
```

```
if (!token) break;
```

```
timestamp = atol(token);
```

```
if (strlen(id) > 0 && port > 0) {
```

```
    NodeDescriptor desc;
```

```
    strcpy(desc.id, id);
```

```
    strcpy(desc.ipaddr, ipaddr);
```

```
    desc.port = port;
```

```
    // Always use current time for freshness
```

```
    desc.timestamp = time(NULL);
```

```
    received[received_count++] = desc;
```

```
}
```

```
}
```

```
// Add received descriptors to my view (excluding myself)
```

```
int added = 0;
```

```
for (int i = 0; i < received_count; i++) {
```

```
    if (strcmp(received[i].id, myDescriptor.id) != 0) {
```

```
        if (add_descriptor(&myView, received[i])) {
```

```
            added++;
```

```
    }  
  }  
}
```

```
printf("→ Added %d descriptors to my view\n", added);
```

```
// Add the last partner back with a fresh timestamp
```

```
last_partner.timestamp = time(NULL);
```

```
update_descriptor(&myView, last_partner);
```

```
} else {
```

```
    // Regular gossip message
```

```
    printf("\n[GOSSIP RECEIVED] %s\n", buf);
```

```
// Check if we've seen this message before
```

```
if (!is_duplicate_message(buf, cached_msgs, &msg_count)) {
```

```
    // Forward to random peers
```

```
    if (myView.count > 0) {
```

```
        // Shuffle view indices for random selection
```

```
        int indices[VIEW_LENGTH];
```

```
        for (int i = 0; i < myView.count; i++) {
```

```
            indices[i] = i;
```

```
        }
```

```
// Fisher-Yates shuffle
```

```
for (int i = myView.count - 1; i > 0; i--) {
```

```
    int j = rand() % (i + 1);
```

```
    int temp = indices[i];
```

```

        indices[i] = indices[j];
        indices[j] = temp;
    }

    // Select up to FORWARD_COUNT peers
    int forward_to = (myView.count < FORWARD_COUNT) ?
myView.count : FORWARD_COUNT;

    struct sockaddr_in peeraddr;
    printf("→ Forwarding to peers:\n");

    for (int i = 0; i < forward_to; i++) {
        memset(&peeraddr, 0, sizeof(peeraddr));
        peeraddr.sin_family = AF_INET;
        peeraddr.sin_port =
htons(myView.descriptors[indices[i]].port);
        inet_pton(AF_INET, myView.descriptors[indices[i]].ipaddr,
&peeraddr.sin_addr);

        printf(" → Peer: %s (%s:%d)\n",
            myView.descriptors[indices[i]].id,
            myView.descriptors[indices[i]].ipaddr,
            myView.descriptors[indices[i]].port);

        sendto(srvsock, buf, strlen(buf), 0,
            (struct sockaddr *)&peeraddr, sizeof(peeraddr));
    }
} else {

```

```

        printf("→ No peers in view to forward message to\n");
    }
} else {
    printf("→ Duplicate message, not forwarding\n");
}
}
}

```

// Handle user input

```

if (result > 0 && FD_ISSET(STDIN_FILENO, &tempset)) {
    memset(buf, 0, MAX_BUFFER_SIZE);
    read(STDIN_FILENO, buf, MAX_BUFFER_SIZE);
    buf[strcspn(buf, "\n")] = 0;

    if (strcmp(buf, "BYE") == 0) {
        printf("Exiting...\n");
        break;
    } else if (strcmp(buf, "VIEW") == 0) {
        // Print current view
        printf("\n[VIEW] Current view (%d nodes):\n", myView.count);
        for (int i = 0; i < myView.count; i++) {
            printf(" %d. %s (%s:%d) [age: %lds]\n",
                i+1,
                myView.descriptors[i].id,
                myView.descriptors[i].ipaddr,
                myView.descriptors[i].port,
                time(NULL) - myView.descriptors[i].timestamp);
        }
    }
}

```

```

    }
} else if (strcmp(buf, "CYCLE") == 0) {
    // Force a Cyclon cycle

    last_cycle_time = 0; // This will trigger a cycle on next iteration
} else {
    // Regular gossip message

    char formattedMessage[MAX_BUFFER_SIZE];

    snprintf(formattedMessage, MAX_BUFFER_SIZE, "%s: %.900s",
myDescriptor.id, buf);

    printf("\n[GOSSIP SENT] %s\n", formattedMessage);

    // Add to cached messages to avoid receiving our own message back
    is_duplicate_message(formattedMessage, cached_msgs,
&msg_count);

    // Select random nodes from view to send to
    if (myView.count > 0) {
        // Shuffle view indices for random selection
        int indices[VIEW_LENGTH];
        for (int i = 0; i < myView.count; i++) {
            indices[i] = i;
        }

        // Fisher-Yates shuffle
        for (int i = myView.count - 1; i > 0; i--) {
            int j = rand() % (i + 1);
            int temp = indices[i];

```

```

        indices[i] = indices[j];
        indices[j] = temp;
    }

    // Select up to FORWARD_COUNT peers
    int  send_to  = (myView.count < FORWARD_COUNT) ?
myView.count : FORWARD_COUNT;

    struct sockaddr_in peeraddr;
    printf("→ Sending to peers:\n");

    for (int i = 0; i < send_to; i++) {
        memset(&peeraddr, 0, sizeof(peeraddr));
        peeraddr.sin_family = AF_INET;
        peeraddr.sin_port = htons(myView.descriptors[indices[i]].port);
        inet_pton(AF_INET,      myView.descriptors[indices[i]].ipaddr,
&peeraddr.sin_addr);

        printf("  → Peer: %s (%s:%d)\n",
            myView.descriptors[indices[i]].id,
            myView.descriptors[indices[i]].ipaddr,
            myView.descriptors[indices[i]].port);

        sendto(srvsock, formattedMessage, strlen(formattedMessage), 0,
            (struct sockaddr *)&peeraddr, sizeof(peeraddr));
    }
} else {
    printf("→ No peers in view to send message to\n");
}

```

```
        }  
    }  
}  
}  
close(srvsock);  
return 0;  
}
```

COMMANDS USED IN THE EXECUTION FOR ONE PEER:

- 1. gcc gossip_cyclon.c -o cyc**
- 2. ./cyc 5001 | tee Alice.log**

OUTPUT:

Alice.log:

```
sakshee@LAPTOP-7PLB9TI2: ~$ cat Alice.log
Node Alice initialized with 3 nodes in view
Initial view contents:
  1. Harry (127.0.0.1:5005)
  2. Susan (127.0.0.1:5004)
  3. Bob (127.0.0.1:5001)

[CYCLON CYCLE] Initiating gossip exchange
→ Selected gossip partner: Harry:5005
→ Sending 2 descriptors to Harry

[CYCLON CYCLE] Initiating gossip exchange
→ Selected gossip partner: Bob:5001
→ Sending 1 descriptors to Bob

[CYCLON RECEIVED] Exchange reply
→ Added 2 descriptors to my view

[CYCLON RECEIVED] Exchange request
→ Added 2 descriptors to my view
→ Replying with 2 descriptors

[CYCLON CYCLE] Initiating gossip exchange
→ Selected gossip partner: Bob:5001
→ Sending 2 descriptors to Bob

[CYCLON RECEIVED] Exchange reply
→ Added 2 descriptors to my view
```

→ Sending 2 descriptors to Bob

[CYCLON RECEIVED] Exchange reply
→ Added 1 descriptors to my view

[CYCLON CYCLE] Initiating gossip exchange
→ Selected gossip partner: Charlie:5002
→ Sending 2 descriptors to Charlie

[CYCLON RECEIVED] Exchange reply
→ Added 2 descriptors to my view

[GOSSIP SENT] Alice: Hello
→ Sending to peers:
 → Peer: Dave (127.0.0.1:5003)
 → Peer: Charlie (127.0.0.1:5002)

[GOSSIP RECEIVED] Alice: Hello
→ Duplicate message, not forwarding

Bob.log:

```
sakshee@LAPTOP-7PLB9TI2: ~$ cat Bob.log
Node Bob initialized with 3 nodes in view
Initial view contents:
  1. Dave (127.0.0.1:5003)
  2. Susan (127.0.0.1:5004)
  3. Charlie (127.0.0.1:5002)

[CYCLON CYCLE] Initiating gossip exchange
→ Selected gossip partner: Dave:5003
→ Sending 2 descriptors to Dave

[CYCLON RECEIVED] Exchange reply
→ Added 2 descriptors to my view

[CYCLON RECEIVED] Exchange request
→ Added 1 descriptors to my view
→ Replying with 2 descriptors

[CYCLON CYCLE] Initiating gossip exchange
→ Selected gossip partner: Susan:5004
→ Sending 2 descriptors to Susan

[CYCLON RECEIVED] Exchange reply
→ Added 2 descriptors to my view
```

```
[CYCLON CYCLE] Initiating gossip exchange
→ Selected gossip partner: Dave:5003
→ Sending 2 descriptors to Dave

[CYCLON RECEIVED] Exchange reply
→ Added 1 descriptors to my view

[GOSSIP RECEIVED] Alice: Hello
→ Forwarding to peers:
  → Peer: Alice (127.0.0.1:5000)
  → Peer: Dave (127.0.0.1:5003)
```

Charlie.log:

```
sakshee@LAPTOP-7PLB9TI2: ~$ cat Charlie.log
Node Charlie initialized with 3 nodes in view
Initial view contents:
  1. Dave (127.0.0.1:5003)
  2. Harry (127.0.0.1:5005)
  3. Susan (127.0.0.1:5004)

[CYCLON CYCLE] Initiating gossip exchange
→ Selected gossip partner: Dave:5003
→ Sending 2 descriptors to Dave

[CYCLON RECEIVED] Exchange reply
→ Added 1 descriptors to my view

[CYCLON CYCLE] Initiating gossip exchange
→ Selected gossip partner: Susan:5004
→ Sending 2 descriptors to Susan

[CYCLON RECEIVED] Exchange reply
→ Added 2 descriptors to my view

[CYCLON RECEIVED] Exchange request
→ Added 1 descriptors to my view
→ Replying with 2 descriptors

[CYCLON CYCLE] Initiating gossip exchange
→ Selected gossip partner: Bob:5001
→ Sending 2 descriptors to Bob
```

```
[CYCLON RECEIVED] Exchange reply
→ Added 1 descriptors to my view

[CYCLON CYCLE] Initiating gossip exchange
→ Selected gossip partner: Charlie:5002
→ Sending 2 descriptors to Charlie

[CYCLON RECEIVED] Exchange reply
→ Added 2 descriptors to my view

[GOSSIP SENT] Alice: Hello
→ Sending to peers:
  → Peer: Dave (127.0.0.1:5003)
  → Peer: Charlie (127.0.0.1:5002)
```

```
[GOSSIP RECEIVED] Alice: Hello
```

Dave.log:

```
sakshee@LAPTOP-7PLB9TI2: . × + ∨  
sakshee@LAPTOP-7PLB9TI2:~$ cat Dave.log  
Node Dave initialized with 3 nodes in view  
Initial view contents:  
  1. Susan (127.0.0.1:5004)  
  2. Charlie (127.0.0.1:5002)  
  3. Harry (127.0.0.1:5005)  
  
[CYCLON RECEIVED] Exchange request  
→ Added 2 descriptors to my view  
→ Replying with 2 descriptors  
  
[CYCLON RECEIVED] Exchange request  
→ Added 2 descriptors to my view  
→ Replying with 2 descriptors  
  
[CYCLON CYCLE] Initiating gossip exchange  
→ Selected gossip partner: Susan:5004  
→ Sending 2 descriptors to Susan  
  
[CYCLON RECEIVED] Exchange reply  
→ Added 2 descriptors to my view  
  
[CYCLON RECEIVED] Exchange request  
→ Added 2 descriptors to my view  
→ Replying with 2 descriptors  
  
[CYCLON CYCLE] Initiating gossip exchange  
→ Selected gossip partner: Bob:5001  
→ Sending 2 descriptors to Bob  
  
[CYCLON RECEIVED] Exchange reply  
→ Added 2 descriptors to my view
```

[CYCLON CYCLE] Initiating gossip exchange

→ Selected gossip partner: Bob:5001

→ Sending 2 descriptors to Bob

[CYCLON RECEIVED] Exchange reply

→ Added 1 descriptors to my view

[CYCLON RECEIVED] Exchange request

→ Added 2 descriptors to my view

→ Replying with 2 descriptors

[GOSSIP RECEIVED] Alice: Hello

→ Forwarding to peers:

→ Peer: Charlie (127.0.0.1:5002)

→ Peer: Harry (127.0.0.1:5005)

Susan.log:

```
sakshee@LAPTOP-7PLB9TI2: ~$ cat Susan.log
Node Susan initialized with 3 nodes in view
Initial view contents:
  1. Dave (127.0.0.1:5003)
  2. Bob (127.0.0.1:5001)
  3. Alice (127.0.0.1:5000)

[CYCLON RECEIVED] Exchange request
→ Added 1 descriptors to my view
→ Replying with 2 descriptors

[CYCLON CYCLE] Initiating gossip exchange
→ Selected gossip partner: Dave:5003
→ Sending 2 descriptors to Dave

[CYCLON RECEIVED] Exchange reply
→ Added 2 descriptors to my view

[CYCLON RECEIVED] Exchange request
→ Added 1 descriptors to my view
→ Replying with 2 descriptors

[CYCLON RECEIVED] Exchange request
→ Added 2 descriptors to my view
→ Replying with 2 descriptors
```

[CYCLON RECEIVED] Exchange request

→ Added 2 descriptors to my view

→ Replying with 2 descriptors

[CYCLON CYCLE] Initiating gossip exchange

→ Selected gossip partner: Charlie:5002

→ Sending 2 descriptors to Charlie

[CYCLON RECEIVED] Exchange reply

→ Added 1 descriptors to my view

[GOSSIP RECEIVED] Alice: Hello

→ Forwarding to peers:

→ Peer: Charlie (127.0.0.1:5002)

→ Peer: Bob (127.0.0.1:5001)

Harry.log:

```
sakshee@LAPTOP-7PLB9TI2: ~$ cat Harry.log
Node Harry initialized with 3 nodes in view
Initial view contents:
  1. Alice (127.0.0.1:5000)
  2. Bob (127.0.0.1:5001)
  3. Charlie (127.0.0.1:5002)

[CYCLON CYCLE] Initiating gossip exchange
→ Selected gossip partner: Alice:5000
→ Sending 2 descriptors to Alice

[CYCLON RECEIVED] Exchange reply
→ Added 1 descriptors to my view

[CYCLON CYCLE] Initiating gossip exchange
→ Selected gossip partner: Bob:5001
→ Sending 2 descriptors to Bob

[CYCLON RECEIVED] Exchange reply
→ Added 2 descriptors to my view

[CYCLON RECEIVED] Exchange request
→ Added 2 descriptors to my view
→ Replying with 2 descriptors

[CYCLON CYCLE] Initiating gossip exchange
→ Selected gossip partner: Alice:5000
→ Sending 2 descriptors to Alice
```

```
[CYCLON RECEIVED] Exchange request
→ Added 2 descriptors to my view
→ Replying with 2 descriptors

[CYCLON CYCLE] Initiating gossip exchange
→ Selected gossip partner: Susan:5004
→ Sending 2 descriptors to Susan

[CYCLON RECEIVED] Exchange reply
→ Added 1 descriptors to my view

[GOSSIP RECEIVED] Alice: Hello
→ Forwarding to peers:
  → Peer: Alice (127.0.0.1:5000)
  → Peer: Bob (127.0.0.1:5001)
```

Key things to note in the output:

1. Gossip exchange happens with peers which weren't initially in the view of a node. This shows that swapping of the descriptors happen successfully.
2. In the STDIN in case message is typed it is sent to all the peers in the network successfully and message forwarding takes place efficiently.
3. In the case of duplication of messages (since redundancy can't be 100 percent reduced in Cyclon though it is much better than the traditional gossip systems), there is efficient mechanism to discard the messages which is also shown in the log outputs.

The users.txt file is as follows:

Alice 127.0.0.1 5000

Bob 127.0.0.1 5001

Charlie 127.0.0.1 5002

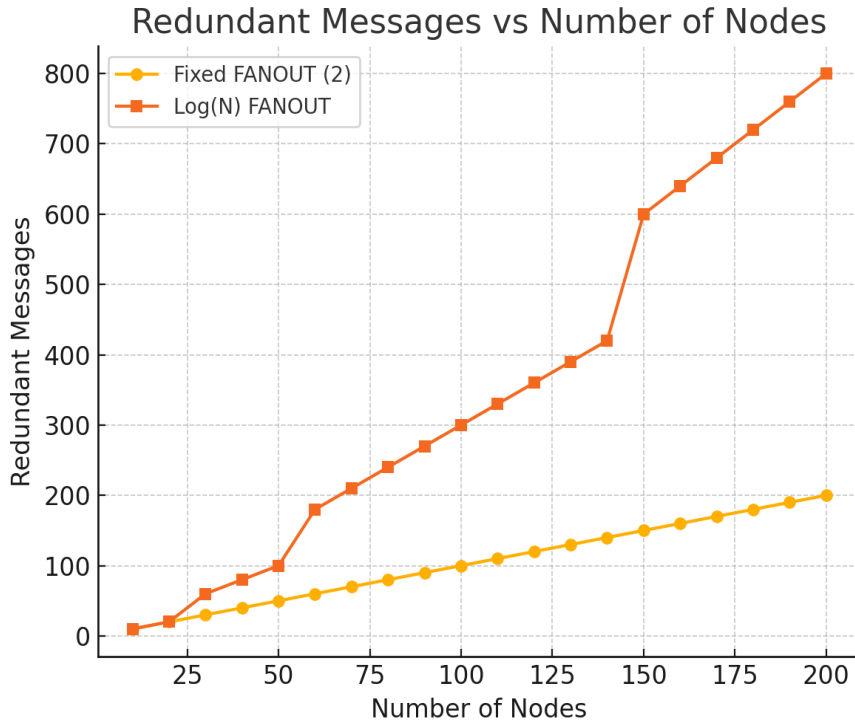
Dave 127.0.0.1 5003

Susan 127.0.0.1 5004

Harry 127.0.0.1 5005

COMPARATIVE ANALYSIS

Traditional gossip protocol:



This graph shows the number of redundant messages as the number of nodes increases when the fanout is fixed to 2 and when the fanout is dynamically set to $\log(N)$ where N represents the number of nodes.

In the case of a fixed fanout of 2:

The number of redundant messages approximately equal N . Here every node send message to 2 other randomly selected nodes, so total messages is $2N$, but in a network only $N-1$ messages are required to fully disseminate the message, so the number of redundant messages is $2N-(N-1)=N+1$. This can be approximated to N .

Hence $R_{\text{fixedfan2}} = N$

In the case of a dynamic fanout of $\log_2(N)$:

Here the total messages is $(\log_2 N) * N$ since every node sends message to other randomly selected $\log_2 N$ nodes, but the number of valid messages in a network is $N-1$ given N nodes in the system. So the number of redundant messages is equal to $N \log_2 N - (N-1)$ which is approximated to $0.73 * N * \log_2 N$.

Hence $R_{\text{variablefanout}} = 0.73 * N * \log_2 N$

The constant 0.73 was determined by fitting the model using linear regression from the graph obtained using a python script.

Reachability vs Redundancy

Here it's important to note that though a fixed fanout of 2 can give us low redundancy but the reachability is very low. Since the selection of peers happen randomly and without the appropriate seeding the same nodes might keep getting selected ending the cycle without all the nodes receiving the message. This is more clearly and prominently observed as the number of nodes increase. Dynamic fanout of $\log_2 N$ allows for good reachability of messages but at the cost of increased redundancy. Hence we require a mechanism which can balance both equally and that is where the Cyclon Gossip comes into the picture. In this reachability is high, redundancy is comparatively reduced and on top of that the message transmission speed is also pretty high.

Intro to Cyclon

Cyclon is a widely-adopted peer-sampling protocol designed to construct and maintain random overlay networks in a fully decentralized and self-organizing manner. Each node in the system maintains a small, fixed-size partial view of other nodes, referred to as descriptors, which contain essential metadata such as the node's identifier, network address, and a timestamp indicating the descriptor's creation time.

In each gossip cycle, a node selects the **oldest descriptor** in its view and initiates a **gossip exchange** with the corresponding peer. During this exchange, both nodes reciprocally send a subset of descriptors, including a fresh descriptor of themselves, effectively replacing old entries with newer and randomly selected ones. This process results in continuous reconfiguration of overlay links and promotes randomness in peer connections.

Cyclon exhibits several desirable properties, including **Scalability**, through bounded local state per node, **High reachability**, by promoting uniform dissemination paths and **Robustness to node failures**, as randomization mitigates the impact of individual departures.

Furthermore, the protocol maintains **bounded indegree** dynamics: while a node increases its outdegree deterministically through self-insertion in each cycle, its indegree is influenced by how frequently it is selected as a gossip partner. This feedback loop leads to a stable equilibrium around the configured view size.

Traditional gossip protocol vs Cyclon Gossip and how Cyclon Gossip is better

Traditional gossip protocols rely on randomized message dissemination, where each node periodically selects a subset of peers and forwards messages (or updates) to them. These protocols are praised for their simplicity, fault-tolerance, and scalability. Typically, in a network of N nodes, each node gossips to a fixed number (fanout) or $\log_2 N$ peers to ensure high reachability and fast message spread.

However, traditional gossip protocols suffer from several **limitations**:

- **Redundant Message Overhead:** To ensure high reachability, traditional gossip often requires a high fanout, leading to substantial message redundancy and unnecessary network traffic.
- **Unbounded View Maintenance:** Nodes do not maintain structured or bounded information about the rest of the network, which can degrade the quality of peer sampling and lead to poor overlay connectivity over time.
- **Inefficient Resource Usage:** Without structured peer sampling, some nodes may be contacted frequently while others are neglected, resulting in uneven load distribution and slower convergence in large systems.

Cyclon, by contrast, is a **structured gossip-based peer sampling protocol** that introduces mechanisms for efficient view management and overlay maintenance. It improves upon traditional gossip in several key ways:

- **Bounded Partial Views:** Each node maintains a fixed-size view (of size ℓ) of descriptors, allowing for predictable resource usage and efficient memory management.
- **Age-Based View Replacement:** Cyclon uses descriptor timestamps to prioritize older entries for replacement, ensuring timely refresh of connections and preventing stale links.
- **Bi-Directional Gossip Exchanges:** In each cycle, nodes engage in mutual exchanges of descriptors. This not only updates both participants' views but also increases the overall randomness and connectivity of the overlay.
- **Overlay Stabilization and Randomness:** Through systematic view updates and descriptor exchanges, Cyclon maintains an overlay with high

randomness, akin to random graphs. This ensures robustness and high connectivity even under churn.

- **Balanced In-Degree and Out-Degree:** Cyclon self-stabilizes toward a state where each node has both in-degree and out-degree close to ℓ , avoiding the skewed distributions observed in traditional gossip overlays.

Reduction in Redundancy using Cyclon:

One of the significant advantages of Cyclon over traditional gossip protocol lies in its ability to reduce message redundancy while preserving high reachability. In traditional gossip, nodes often forward messages to multiple randomly selected peers without awareness of the overall network state, leading to excessive redundant transmissions especially as the number of nodes increases. Cyclon addresses this inefficiency through its structured peer-sampling mechanism, where each node maintains a fixed-size, constantly refreshed partial view of the network. Instead of broadcasting messages to random peers, Cyclon nodes engage in targeted, pairwise descriptor exchanges, effectively limiting the number of messages disseminated in each cycle. This controlled and selective interaction ensures that messages propagate efficiently across the network without the uncontrolled flooding seen in classical gossip approaches. Moreover, by refreshing the view based on descriptor age, Cyclon promotes diversity in peer interactions, reducing the likelihood of repeatedly contacting the same nodes, which is a major contributor to redundancy in traditional systems. As a result, Cyclon achieves a more optimal balance between coverage and communication overhead, minimizing redundancy without compromising on network connectivity or robustness.

Conclusion:

Hence, Cyclon refines the traditional gossip approach by integrating structured view management, randomized descriptor exchange, and self-healing overlay formation. This results in a **scalable**, **efficient**, and **resilient** gossip-based protocol well-suited for large-scale, dynamic, and decentralized systems.

REFERENCES

- [1] S. Voulgaris, D. Gavidia, and M. van Steen,
**"CYCLON: Inexpensive Membership Management for Unstructured P2P
Overlays,"**
Journal of Network and Systems Management, vol. 13, no. 2, pp. 197–217, June
2005.
- [2] A. Antonov and S. Voulgaris,
"SecureCyclon: Dependable Peer Sampling,"
in *2023 IEEE 43rd International Conference on Distributed Computing Systems
(ICDCS)*,
2023, pp. 380–391. doi: 10.1109/ICDCS57875.2023.00041